

Object Oriented Programming | Java Method

1. What is a Java Method?

A method in Java is a group of statements that are used to perform a specific task. A method must be declared inside a class.

```
class Test {  
    void display() {  
        System.out.println("Hello");  
    }  
}
```

It can be called whenever we need to perform that task. Here, `display()` is a method that performs the task of printing `"Hello"`.

2. Types of Methods in Java

In Java, there are two types of methods:

1. Standard Library Methods:

These are built-in methods already provided by Java. We can use them directly.

Examples: `print()`, `nextInt()`, `sqrt()`

2. User-defined Methods:

These are methods created by the programmer according to their requirements.

Examples: `display()`, `sum()`

Key Takeaway:

- **Standard Library Method** → Already provided by Java
- **User-defined Method** → Created by the programmer

3. Syntax of Defining a Method / Function

```
returnType functionName(parameters) {  
    // statements  
}
```

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

4. Components of a Method in Java

- **Modifier:** Defines the access type of the method. (Example: `public` , `private`)
- **Return Type:** Specifies the type of data returned by the method. (Example: `int` , `double` , `void`)
- **Method Name:** The name of the method. (Example: `add`)
- **Parameter List:** Contains the type, order, and number of parameters. (Example: `(int a, int b)`)
- **Body of Method:** Defines what the method does.

5. Identifying Components of a Method

Example Method:

```
public int add(int a, int b) {  
    return a + b;  
}
```

Breakdown of Components:

- **Modifier:** `public`
- **Return Type:** `int`
- **Method Name:** `add`
- **Parameter List:** `(int a, int b)`
- **Body of Method:** `{ return a + b; }`

Explanation: This method takes two integers as parameters and returns their sum.

6. Categories of User-Defined Methods

User-defined methods can be divided into four categories based on arguments (parameters) and return value:

1. Method with No Arguments and No Return Value
2. Method with Arguments but No Return Value
3. Method with No Arguments but With Return Value
4. Method with Arguments and Return Value

7. Java Programs & Code Examples

Program 1: Add two integer values using a static method

```
public class Method1 {  
  
    public static void main(String[] args) {  
        int a = 10, b = 5;  
        sum(a, b);  
    }  
  
    static void sum(int a, int b) {  
        System.out.println("The Sum is: " + (a + b));  
    }  
}
```

Program 2: Find the sum of two integer numbers using a method and creating an object

```
public class Method1 {  
  
    void sum(int a, int b) {  
        System.out.println("The Sum is: " + (a + b));  
    }  
  
    public static void main(String[] args) {  
        int a = 10, b = 5;  
  
        Method1 obj = new Method1();  
        obj.sum(a, b);  
    }  
}
```

Program 3: Find the maximum between two numbers using a method (max()) and an object

```
public class Maximum {

    int num1 = 10;
    int num2 = 20;

    // max() method
    int max(int num1, int num2) {
        if (num1 > num2)
            return num1;
        else
            return num2;
    }

    // Printing method
    void printResult(int result) {
        System.out.println("Maximum = " + result);
    }

    // Main method
    public static void main(String[] args) {

        Maximum obj = new Maximum();

        int result = obj.max(obj.num1, obj.num2);

        obj.printResult(result);
    }
}
```

Program 4: Find the Maximum Between Two Numbers Using a Method Without an Object

```
public class Maximum {  
  
    // max() method  
    static int max(int num1, int num2) {  
        if (num1 > num2)  
            return num1;  
        else  
            return num2;  
    }  
  
    // Printing method  
    static void printResult(int result) {  
        System.out.println("Maximum = " + result);  
    }  
  
    // Main method  
    public static void main(String[] args) {  
  
        int num1 = 10;  
        int num2 = 20;  
  
        // Calling max() method without object  
        int result = max(num1, num2);  
  
        // Calling printing method  
        printResult(result);  
    }  
}
```

Method Overloading

Method overloading means having two or more methods with the same name but different parameters in a class. It makes the program easier to read and understand.